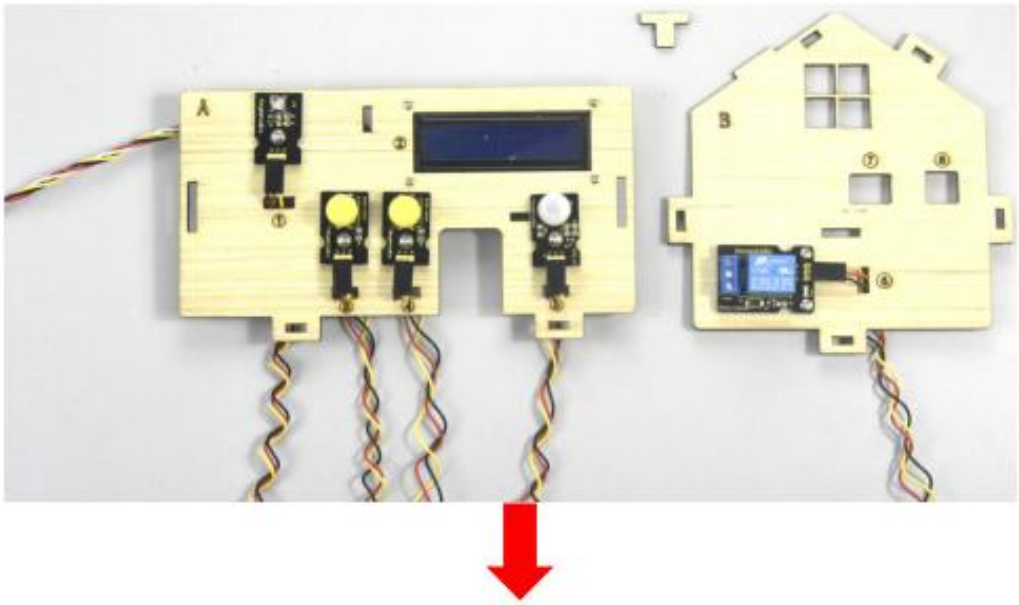
Smart Home Kit for Arduino - Lesson Guide



How to Work Through Each Lesson

Follow the same steps each time. Once you have completed the steps and passed the check, you can move on to the challenges!

1. First, look at the circuit diagram and note which components and pins you need.
2. Disconnect the USB cable before changing any wires. Connect G, V, and S as shown in the diagram, and check each connection.
3. Connect the USB cable, select the Keyestudio / Arduino Uno board and the correct port, then click Upload.
4. Try the challenge by changing one thing at a time. Note what you changed and what you observed.

CONNECTIONS G = ground (GND) • V = power (5V) • S = signal. Always follow the labels shown in the sensor diagram. Do not change connections while the board is powered.

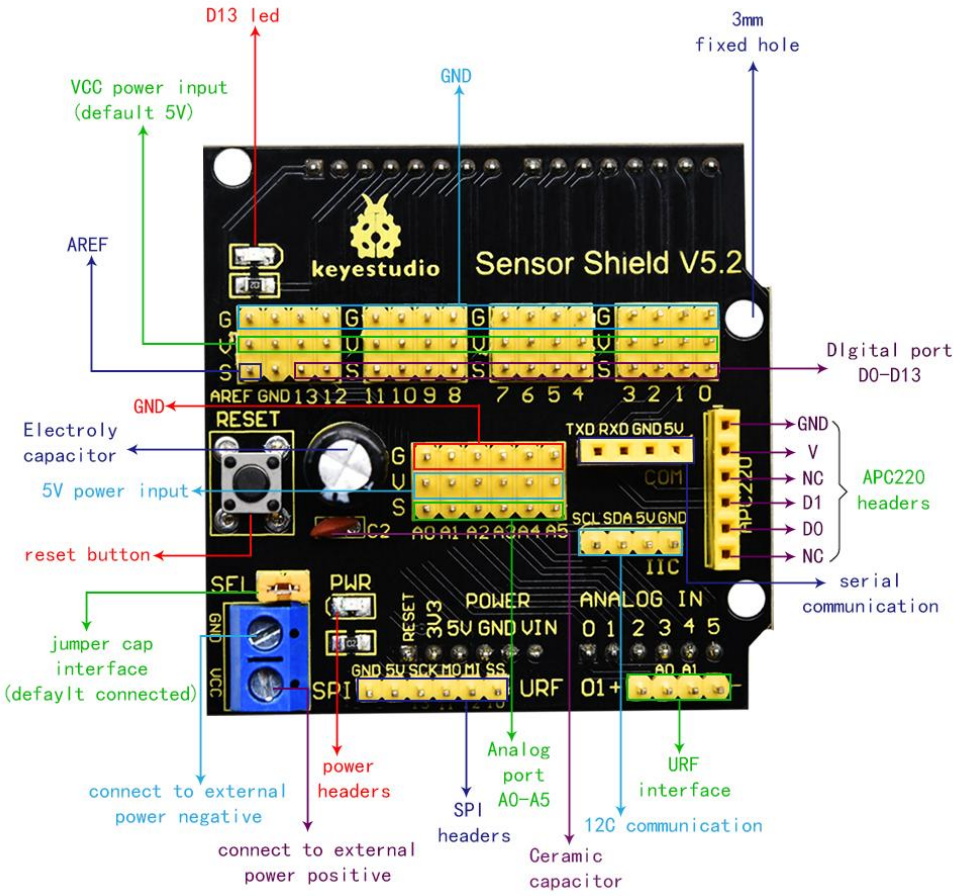
Quick Check Before You Click Upload

- The USB cable was disconnected while I built the circuit.
- The G, V, and S connections match the diagram.
- The correct port and board are selected.
- I saved a copy of the file before editing the code.

Safety Rules

- Use the relay only with the kit's low-voltage circuit. Never connect it to a wall outlet or household mains power.
- The MQ-2 is for classroom experiments, not a safety alarm. Do not test it with a flame, lighter, or household gas.
- Keep water and moisture away from the board. The water sensor is not waterproof.
- Keep fingers, hair, and loose objects away from the fan. Do not force the servo by hand.

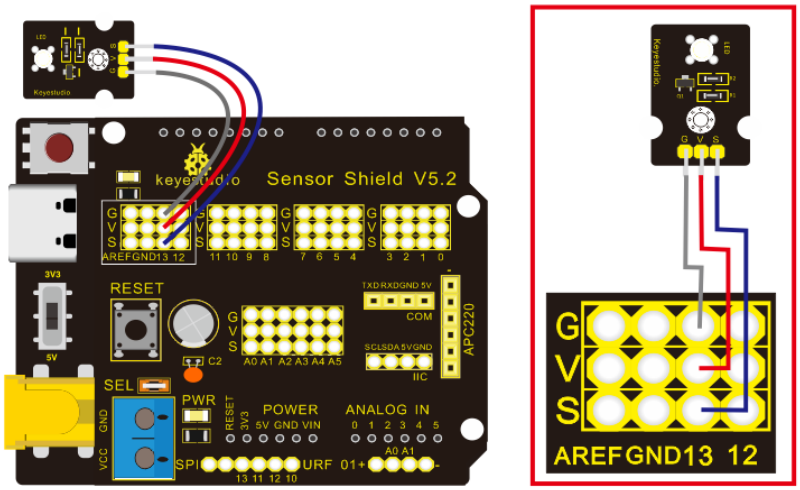
The kit guide contains the full description of each project. Page numbers mentioned in these worksheets refer to the KS0085 PDF.


D13 led
3mm
fixed hole
GND
VCC power input
(default 5V)
AREF
keyestudio
Sensor Shield V5.2
Dlgital port
D0-D13
GND
Electroly
capacitor
5V power input
reset button
RESET
TXD RXD GND 5V
COM
APC220
GND
V
NC
D1
D0
NC
APC220
headers
SCL SDA 5V GND
IIC
serial
communication
SEL
PWR
POWER
ANALOG IN
jumper cap
interface
(defaylt connected)
SPI
URF
connect to external
power negative
power
headers
connect to external
power positive
SPI
headers
Analog
port
A0-A5
Ceramic
capacitor
12C communication
URF
interface

Goal: Connect the LED module to G, V, and D13.

Wiring

Connect the LED module to G, V, and D13.



Wiring diagram for the white LED on output D13.

For 3-pin cables: G = ground • V = 5V • S = signal

Programming

To control the LED, we will use the digitalWrite() function.

The digitalWrite() function sends a digital signal to an Arduino pin, setting its voltage to either HIGH (high voltage, usually 5V or 3.3V) or LOW (0V / ground).

```
digitalWrite(pin, value);
```

- pin: The number of the digital pin you want to control (for example, 13 or A0).
- value: The value to set. It can be HIGH (on) or LOW (off).

Basic Rules and How It Works

- Required first step (pinMode): Before using digitalWrite(), set the pin as an output in setup() by using pinMode(pin, OUTPUT).
- What HIGH does: Sets the pin to the board's high voltage (5V on most Arduino boards such as the Uno, or 3.3V on some newer boards / ESP32) to turn on an LED or activate a relay.
- What LOW does: Sets the voltage to 0V, turning off the connected device.

Example Program

```
const int LED_PIN = 13;
```

```
void setup() {
  pinMode(LED_PIN, OUTPUT);
}
void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(1000);
  digitalWrite(LED_PIN, LOW);
  delay(1000);
}
```

Check

Click Upload and make sure the LED blinks once every second. If it does not work, check:

- That the code was verified and uploaded successfully.
- That the correct COM / ttyUSB port is selected.
- That the connections are correct.

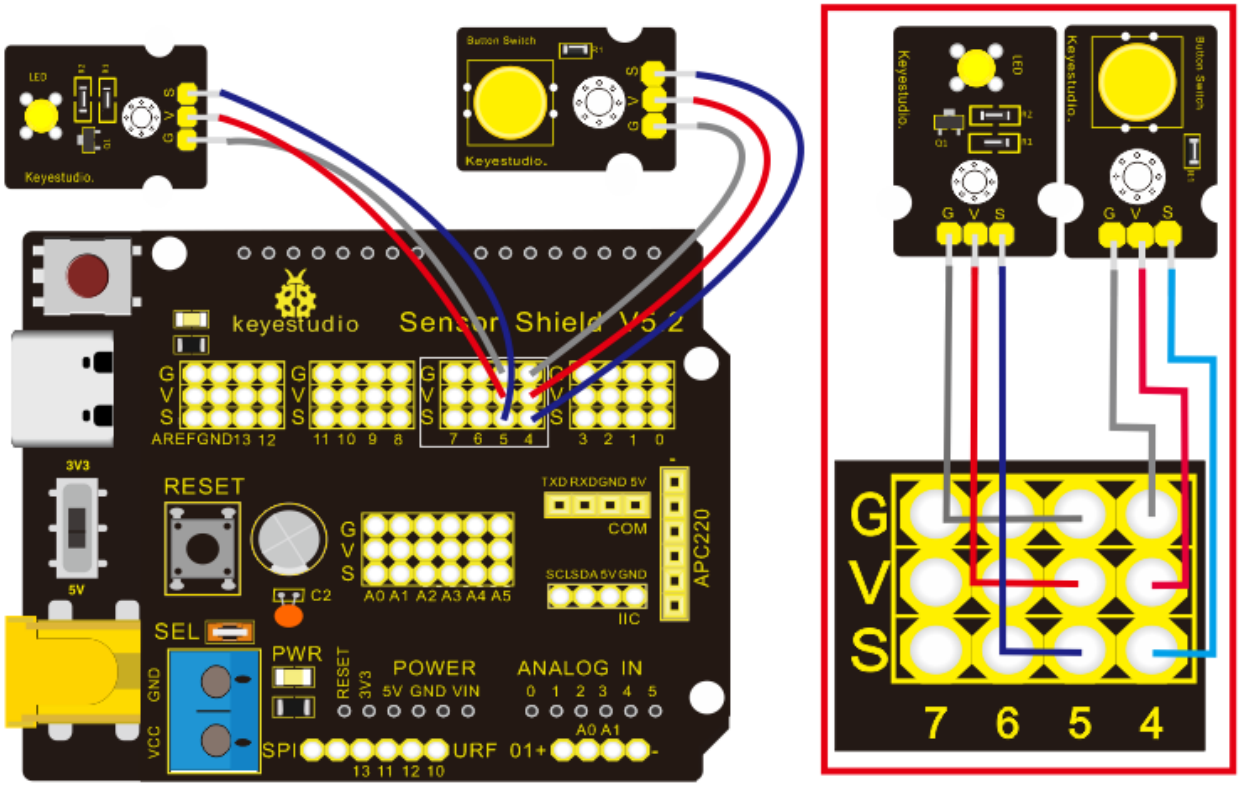
Challenges

1. Change the blinking speed so that the LED appears to stay on continuously.
2. Add a second LED. When the first LED turns on, the second one must turn off.
3. Super challenge! Create a sequence of blinks with two LEDs. Your classmates should watch and copy it! Then switch roles.

Lesson 02 - Button

Wiring

Connect the button module to G, V, and D4, and the LED to D5, as shown in the diagram.



Programming

To check the button state, we will use digitalRead(), the input counterpart to digitalWrite(). The digitalRead() function reads the state of a digital pin: whether it has a voltage (HIGH) or not (LOW).

```
buttonState = digitalRead(pin);
```

- pin (input parameter): The number of the digital pin you want to read. In this example, use 4 (or the buttonPin variable).

- Return value: The function returns one of two possible states, depending on the voltage detected at the pin at that moment:
    - HIGH: The pin has a high voltage (5V or 3.3V). This usually means the button is pressed.
    - LOW: The pin has no voltage (0V / ground). This means the button is released.

Basic Rules and How It Works

- Required first step (pinMode): Before reading the pin, set it as an input in setup() using pinMode(4, INPUT).
- Storing the value: Since digitalRead() returns a value (HIGH or LOW), store it in a variable (for example, buttonState) so that you can check it later with an if statement.

Example Program 1

```
const int buttonPin = 4; // Button connected to Pin 4
int buttonState = 0; // Variable for storing HIGH or LOW

void setup() {
  pinMode(buttonPin, INPUT); // Set Pin 4 as an input (reads)
  Serial.begin(9600); // Start communication with the computer
}

void loop() {
  // Read Pin 4 and store the result (HIGH or LOW)
  buttonState = digitalRead(buttonPin);

  // Display the value on the computer screen
  Serial.print("Button State: ");
  Serial.println(buttonState);

  delay(100); // Short delay so we can read the values
}
```

Check:

1. Click Upload in the Arduino IDE.
2. Make sure the correct COM / ttyUSB port and board are selected.
3. Open the Serial Monitor (the magnifying-glass icon at the top right of the Arduino IDE).
4. What you should see: When the button is not pressed, the value should be 0 (LOW). When you press and hold it, the value should immediately change to 1 (HIGH).

Now that we know the button works, connect the LED to Pin 5 as well. The input (button) will now control the output (LED).

Example Program 2

```
const int buttonPin = 4; // Button connected to Pin 4
```

```
const int ledPin = 5; // LED connected to Pin 5

int buttonState = 0; // Variable for storing HIGH or LOW

void setup() {
  pinMode(buttonPin, INPUT); // Set Pin 4 as an input (reads)
  pinMode(ledPin, OUTPUT); // Set Pin 5 as an output (writes)
}

void loop() {
  // 1. Read Pin 4 and store the result (HIGH or LOW)
  buttonState = digitalRead(buttonPin);

  // 2. Check the value returned by digitalRead()
  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH); // If the reading is HIGH, turn on the LED
  } else {
    digitalWrite(ledPin, LOW); // If the reading is LOW, turn off the LED
  }
}
```

Check

Click Upload and make sure the LED turns on when you press the button and turns off when you release it:

- That the code was verified and uploaded successfully.
- That the correct COM / ttyUSB port is selected.
- That the connections are correct.

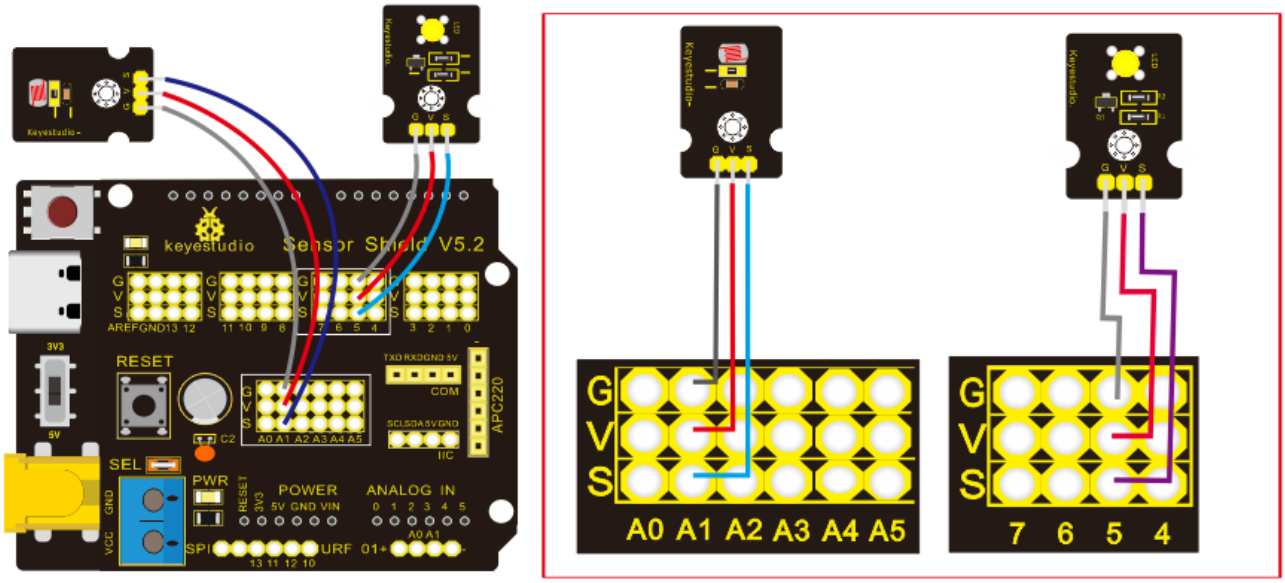
Challenges

1. Make the LED work in reverse: it should be on when the button is not pressed and turn off when the button is pressed.
2. While you hold down the button, make the LED blink quickly, like a car's turn signal.
3. So far, the LED turns off as soon as you release the button. Now make the button work like a room light switch: press and release once to turn the LED on and leave it on; press and release again to turn it off. (Hint: You will also need a short delay(200) so the Arduino is not confused by button bounce.)
4. **When the program starts, the LED is off. The Arduino waits for a random amount of time (for example, 2 to 5 seconds). Suddenly, the LED turns on!**
   Press the button as quickly as you can. If you press it within 300 milliseconds, the LED blinks happily three times (You win!). If you are too late, the LED slowly turns off (You lose!).

Lesson 03 - Measuring Light

Wiring

Connect the LDR (light-dependent resistor) module to G, V, and A0, and the LED to D5, as shown in the diagram.
An LDR is a resistor whose resistance changes with light. Use a multimeter to measure how its value changes under different lighting conditions.



Programming

The analogRead() function reads an analog pin. It converts the voltage from the LDR module into a digital value from 0 to 1023.

```
ldrValue = analogRead(pin);
```

- pin (input parameter): The analog pin number you want to read. In this example, use A0 (or the ldrPin variable).
- Return value: The function returns an integer from 0 to 1023, depending on the light falling on the module at that moment:
    - High value (close to 1023): This corresponds to a high voltage (5V) and usually means there is a lot of light in the room.
    - Low value (close to 0): This corresponds to a low voltage (0V) and usually means it is dark.

Basic Rules and How It Works

- Analog pins: Analog pins (A0-A5) work as inputs automatically, so you do not need to write pinMode(A0, INPUT) in setup().
- Storing the value: Since analogRead() returns a changing value (0-1023), store it in a variable (for example, ldrValue) so you can display it or check it with an if statement.

Example Program 1

```
const int ldrPin = A0; // The midpoint of the voltage divider on analog pin A0
int ldrValue = 0; // Integer variable for storing the light level

void setup() {
  Serial.begin(9600); // Start communication with the computer
}
```

```
void loop() {
  // Read the analog value (0 to 1023)
  ldrValue = analogRead(ldrPin);

  // Display the value on the computer screen
  Serial.print("LDR Value: ");
  Serial.println(ldrValue);

  delay(100); // Short delay so we can read the values
}
```

Check:

1. Click Upload in the Arduino IDE.
2. Make sure the correct COM / ttyUSB port and board are selected.
3. Open the Serial Monitor (the magnifying-glass icon at the top right of the Arduino IDE).
4. What you should see: When you shine light on the sensor, the number should rise toward 1023. When you cover it with your hand, the number should fall toward 0.
5. Take note: Check the value when the sensor is completely covered (for example, 280). This will be your "darkness threshold."

Example Program 2

```
const int ldrPin = A0; // LDR connected to Pin A0
const int ledPin = 5; // LED connected to Pin 5
int ldrValue = 0; // Integer variable for storing the light level

void setup() {
  pinMode(ledPin, OUTPUT); // Set Pin 5 as an output (writes)
}
void loop() {
  // 1. Read Pin A0 and store the result (0 to 1023)
  ldrValue = analogRead(ldrPin);

  // 2. Check whether the value has fallen below 300 (meaning it has become dark)
  if (ldrValue < 300) {
    digitalWrite(ledPin, HIGH); // If it is dark, turn on the LED
  } else {
    digitalWrite(ledPin, LOW); // If it is light, turn off the LED
  }
}
```

Check

Click Upload and make sure the LED turns on automatically when you cover the LDR with your hand and turns off when you uncover it:

1. That the code was verified and uploaded successfully.
2. That the correct COM / ttyUSB port is selected.
3. That the connections are correct.
4. Depending on the room lighting, you may need to change the darkness threshold.

Challenges!

1. Write a program that increases the value of an int variable named dark while it is dark.
   Create the variable at the beginning of the program, before void setup().
   Then, inside void loop(), check whether the light level has dropped (for example, analogRead(A0) < 300). If it has, increase the variable by 1 (dark = dark + 1 or dark++).
2. In the example, the LED turns on immediately. Now make it turn on only if it stays dark for a while, so that a brief shadow or a bird passing in front of the sensor does not turn it on unnecessarily.
3. Build a refrigerator alarm! When the LDR is in darkness (the door is closed), the LED is off. If someone opens the door and light enters, the Arduino should wait 5 seconds (enough time to take something out). If the door stays open longer, make the LED flash rapidly to warn that it was left open. In the next lesson, we will add a buzzer to make this more interesting!

The if and while Statements

The if statement runs commands only when a condition is true. For example, we can turn on an LED when a button is pressed:

The while statement repeats the commands inside the braces { } as long as a condition is true.

```
if (digitalRead(κουμπί) == HIGH) {
  digitalWrite(led, HIGH);
  // Αν πατηθεί το κουμπί, ανάβει το LED
}

while (digitalRead(κουμπί) == HIGH) {
  // Περιμένει όσο το κουμπί παραμένει πατημένο
}

digitalWrite(led, LOW);  // Σβήνει το LED όταν αφεθεί το κουμπί
```

Lesson 04 - Sound!

Wiring

Connect the pins of the Passive Buzzer module directly to the board using jumper wires:

- Connect pin G (GND / ground) to G (GND) on the Arduino.
- Connect pin V (VCC / power) to V (5V) on the Arduino.
- Connect pin S (signal) to digital Pin 3 on the Arduino.

The Passive Buzzer module has the buzzer component built into a small circuit board.
Unlike an active buzzer, a passive buzzer works like a tiny speaker.

- If we supply it with steady power (HIGH), it will not make a sound (perhaps only a faint click).
- To make a sound, we must send it a signal pulsing at a specific frequency, making its internal membrane vibrate.
- Its main advantage is that changing the frequency changes the note. It can therefore play different musical notes and melodies.

To understand how the Arduino makes sounds with a buzzer, or blinks an LED so quickly that it "tricks" our eyes, we need to know two basic ideas: period and frequency.

- Period (T): The total time needed to complete one full cycle of a repeating event (for example, an LED turning on and off). It is measured in seconds (s).
- Frequency (f): The number of times this cycle repeats in one second. It is measured in hertz (Hz).

Their mathematical relationship is:

$$συχνότητα = \frac{1}{περίοδος} \rightarrow f = \frac{1}{T}$$

In the basic code, the LED blinks once per second:

```
digitalWrite(ledPin, HIGH); delay(1000);
digitalWrite(ledPin, LOW);  delay(1000);
```

Question
What do you think the period is?
What is the frequency?

T = ... s f = ... Hz

The Biological Limit of Human Vision
When we reduced the delay() in the code, the LED blinked faster and faster.

- If we set delay(10); (10 ms on and 10 ms off), the total period is 10 + 10 = 20 ms.
- What is the frequency then?
- That means the LED blinks 50 times per second!

The "Trick" of Our Eyes (Flicker Fusion)
The human visual system has a biological speed limit. When a light blinks with a period shorter than about 12-15 ms (corresponding to frequencies above roughly 70-85 Hz), the brain cannot register the gaps when the light is off.

This effect is called flicker fusion. To an observer, the LED appears to stop flickering and stay on continuously (with slightly lower brightness).

Modern technology uses this same effect:

- Mobile phone and computer screens: They refresh at 60 Hz, 90 Hz, or 120 Hz, updating the image quickly enough for graphics and motion to appear smooth and steady.
- Movies: Films show 24 still frames per second. A projector may show each frame two or three times, raising the flicker rate to 48 Hz or 72 Hz, so the motion appears continuous and is easier on the eyes.

With a passive buzzer, instead of tricking the eye by blinking an LED, we make the air vibrate! When we use tone(buzzerPin, 440); (the musical note A), the Arduino switches Pin 3 at a frequency of 440 times per second. This means each vibration has a period of about 2.2 ms.
This very rapid electrical switching makes the buzzer's internal membrane vibrate. The vibration moves the air, producing sound waves that our ears interpret as a musical tone.

Answer

- Period: One full cycle (on + off) takes 1000 ms + 1000 ms = 2000 ms, or 2 seconds. Therefore, T = 2 s.
- Frequency: Only half a cycle occurs in one second. Dividing the number of cycles by time gives a frequency of 0.5 Hz.

Programming: Sound Functions

To control the notes, frequency, and duration of sound from the Passive Buzzer module, Arduino provides two useful built-in functions:

A) The tone() Function

This function generates a square-wave signal at the output pin, making the buzzer membrane vibrate and produce a specific musical note.

```
tone(pin, frequency);
// or
tone(pin, frequency, duration);
```

- pin: The Arduino digital pin that sends the signal to the module (Pin 3 in this lesson).
- frequency: The sound frequency in hertz (Hz). A lower number produces a lower-pitched sound; a higher number produces a higher-pitched sound (for example, 262 is note C and 440 is note A).
- duration (optional): How long the sound lasts, in milliseconds.

B) The noTone() Function

Stops sound on the specified pin immediately. If tone() is used without a duration, the buzzer keeps sounding until noTone() is called.

```
noTone(pin);
```

Example Program 1

This code continually changes the frequency sent to the module on Pin 3, creating a changing sound like an ambulance siren.

```
const int buzzerPin = 3; // Passive Buzzer signal on Pin 3

void setup() {
 pinMode(buzzerPin, OUTPUT); // Set Pin 3 as an output
}

void loop() {
 tone(buzzerPin, 440); // Play 440 Hz (note A)
 delay(500); // Hold the sound for half a second
 tone(buzzerPin, 880); // Play 880 Hz (one octave higher)
 delay(500); // Hold the sound for half a second
}
```

Check:

1. Click Upload in the Arduino IDE.
2. Make sure the correct COM / ttyUSB port is selected.
3. What you should hear: The buzzer should automatically switch between the two sounds every half second without stopping.

Example Program 2: Sound with a Pause (Beep-Blink or Blink-Beep)

This code makes a short beep, then uses noTone() to create a clear one-second pause.

```
const int buzzerPin = 3; // Το σήμα του Passive Buzzer στο Pin 3

void setup() {
  pinMode(buzzerPin, OUTPUT); // Ορισμός του Pin 3 ως Έξοδο
}

void loop() {
  tone(buzzerPin, 1000); // Παίζει έναν καθαρό ήχο στα 1000Hz
  delay(150);            // Αφήνει τον ήχο να ακουστεί για 150ms
  noTone(buzzerPin);     // Σταματάει τον ήχο τελείως
  delay(1000);           // Περιμένει 1 δευτερόλεπτο στη σιωπή πριν το επόμενο μπιπ
}
```

Check:

1. Click Upload in the Arduino IDE.

2. What you should hear: The buzzer should make a short beep once every second, like an alarm clock.

Challenge 1

- Change the code so that the buzzer does not just beep continuously, but plays the international Morse code distress signal SOS. SOS consists of three short beeps, three long beeps, and three short beeps.

Challenge 2

- Create a countdown sound effect! The buzzer should make short beeps that get closer and closer together (for example, start with a 1000 ms delay between beeps and gradually reduce it to 0, then play one long "explosion" sound).
  First, read the explanation and examples of the for statement.

The for Statement

The for statement repeats commands several times. We use a counter to keep track of how many times the code runs.

```
for (int i = 0; i < 5; i++) {
 // commands to be repeated
}
```

Here:

- int i = 0 starts counting at 0.
- i < 5 means the loop continues as long as i is less than 5.
- i++ increases i by 1 each time.

The value of i changes like this:

Repetition	Value of i
1st	0
2nd	1
3rd	2
4th	3
5th	4
6th	5 (loop ends)

Example:

```
void setup() {
  pinMode(ledPin, OUTPUT);

  for (int delayTime = 100; delayTime >= 1000; delayTime += 100) {
    digitalWrite(ledPin, HIGH);
    delay(delayTime);
    digitalWrite(ledPin, LOW);
    delay(delayTime);
  }
}
```

In this example, we change the delay time in steps of 100 ms. It starts at 100 ms and increases by 100 ms each time until it reaches 1000 ms.
If we wrote:

```
  for (int delayTime = 1000; delayTime <= 100; delayTime -= 100) {
```

we would get the opposite. The delay starts at 1000 ms and decreases by 100 ms each time until it reaches 0.

Challenge 3: "Theremin - Music with Shadows" (Boss Level)
Goal: Combine the LDR module (Pin A0) from the previous lesson with the Buzzer module (Pin 3).
Build a futuristic musical instrument that changes its sound without being touched! As you move your hand closer to the LDR and block the light, the buzzer's pitch should change automatically, becoming lower or higher.
Hint: Use analogRead(A0) and map() to convert the light values (0-1023) directly into sound frequencies (for example, 150 to 1500 Hz) for tone().

The map() Function

The map() function converts a number (value) from one range to another. That is:

1. the value fromLow maps to toLow,
2. the value fromHigh maps to toHigh,
3. and the values in between are mapped proportionally.

```
map(value, fromLow, fromHigh, toLow, toHigh)
```

Example

```
void loop() {
  int val = analogRead(0);

  Serial.print("Input value: ");
  Serial.print(val);

 val = map(val, 0, 1023, 0, 255); // mapping function

  Serial.print(", Output value: ");
  Serial.println(val);
```

The variable val will have a value from 0 to 1023.
These values are then mapped to the new range, from 0 to 255.
Exercise
Fill in the blanks by writing the value from the example that corresponds to each map() parameter.

```
val = map(val, 0, 1023, 0, 255);
```

- toHigh = _____
- value = _____
- fromHigh = _____
- fromLow = _____
- toLow = _____

Lesson 4.5 - Making Music

Challenge 1: “Make a Piano!”

Goal: Connect three button modules (for notes C, D, and E) and the Passive Buzzer module. When you press a button, the buzzer should play its corresponding frequency from the list (C = 262 Hz, D = 294 Hz, E = 330 Hz). When you release it, the sound should stop.

```
#define NOTE_C 262
#define NOTE_D 294
#define NOTE_E 330
..
..
..
const int buzzerPin = 3;
const int btnC = 4;
const int btnD = 6;
const int btnE = 7;

void setup() {
  pinMode(buzzerPin, OUTPUT);
  pinMode(btnC, INPUT);
  pinMode(btnD, INPUT);
  pinMode(btnE, INPUT);
}

void loop() {
 // Check whether the first button (C) is pressed
  if (digitalRead(btnC) == HIGH) {
    tone(buzzerPin, NOTE_C);
  }
...
  else {
    noTone(buzzerPin);
  }
}
```

NOTE FREQUENCY CHART | HEROIC AUDIO

	Octave 0	Octave 1	Octave 2	Octave 3	Octave 4	Octave 5	Octave 6	Octave 7	Octave 8	Octave 9	Octave 10
C	16.35	32.70	65.41	130.81	261.63	523.25	1046.50	2093.00	4186.01	8372.02	16744.04
C#	17.32	34.65	69.30	138.59	277.18	554.37	1108.73	2217.46	4434.92	8869.84	17739.69
D	18.35	36.71	73.42	146.83	293.66	587.33	1174.66	2349.32	4698.64	9397.27	18794.55
D#	19.45	38.89	77.78	155.56	311.13	622.25	1244.51	2489.02	4978.03	9956.06	19912.13
E	20.60	41.20	82.41	164.81	329.63	659.26	1318.51	2637.02	5274.04	10548.08	
F	21.83	43.65	87.31	174.61	349.23	698.46	1396.91	2793.83	5587.65	11175.30	
F#	23.12	46.25	92.50	185.00	369.99	739.99	1479.98	2959.96	5919.91	11839.82	
G	24.50	49.00	98.00	196.00	392.00	783.99	1567.98	3135.96	6271.93	12543.86	
G#	25.96	51.91	103.83	207.65	415.30	830.61	1661.22	3322.44	6644.88	13289.75	
A	27.50	55.00	110.00	220.00	440.00	880.00	1760.00	3520.00	7040.00	14080.00	
A#	29.14	58.27	116.54	233.08	466.16	932.33	1864.66	3729.31	7458.62	14917.24	
B	30.87	61.74	123.47	246.94	493.88	987.77	1975.53	3951.07	7902.13	15804.26	

Challenge 2: “Make a Synthesizer!”

Goal: Use for loops to create different sounds. Complete the program outline so that each button activates its own sound effect.

- Button 1 (Pin 4) - Laser effect: The frequency starts at 1000 Hz and gradually falls to 200 Hz.
- Button 2 (Pin 5) - Siren effect: The frequency starts at 500 Hz, rises to 1200 Hz, then falls back to 500 Hz.
- Button 3 (Pin 6) - Rising effect: The frequency starts at 200 Hz and rises to 1000 Hz.

Change the frequency by 10 Hz in each repetition. Use delay(5) so each frequency is heard briefly.

```
const int buzzerPin = 3;
const int btnLaser = 4;
const int btnSiren = 5;
const int btnRise = 6;

void setup() {
  pinMode(buzzerPin, OUTPUT);
  pinMode(btnLaser, INPUT);
  pinMode(btnSiren, INPUT);
  pinMode(btnRise, INPUT);
}

void loop() {
 // Button 1: Laser effect
  if (digitalRead(btnLaser) == HIGH) {
  // Complete a for loop:
  // Start at 1000 Hz and decrease to 200 Hz.
  // In each repetition:
  // 1. Call tone(buzzerPin, freq)
  // 2. Wait 5 ms using delay(5)
    noTone(buzzerPin);
  }
 // Button 2: Siren effect
  else if (digitalRead(btnSiren) == HIGH) {
  // Complete a for loop that increases the frequency
  // from 500 Hz to 1200 Hz.

  // Complete a for loop that decreases the frequency
  // from 1200 Hz to 500 Hz.
    noTone(buzzerPin);
  }
 // Button 3: Rising effect
  else if (digitalRead(btnRise) == HIGH) {
  // Complete a for loop:
  // Start at 200 Hz and increase to 1000 Hz.
    noTone(buzzerPin);
  }
}
```

Challenge 3

Explore the contents of the repository below.

https://github.com/robsoncouto/arduino-songs

Theremin Solution

```
const int ldrPin = A0;     // Το LDR Module στην Αναλογική θύρα A0
const int buzzerPin = 3;   // Το Passive Buzzer Module στο Pin 3

int ldrValue = 0;          // Μεταβλητή για την αποθήκευση της τιμής του φωτός
int frequency = 0;         // Μεταβλητή για τη συχνότητα του ήχου

void setup() {
  pinMode(buzzerPin, OUTPUT); // Ορισμός του Pin 3 ως Έξοδο
  Serial.begin(9600);         // Ενεργοποίηση Serial Monitor για έλεγχο
}

void loop() {
  // 1. Διαβάζουμε την αναλογική τιμή του φωτός (από 0 έως 1023)
  ldrValue = analogRead(ldrPin);

  // 2. Η ΣΥΝΑΡΤΗΣΗ-ΚΛΕΙΔΙ: Μετατρέπουμε το εύρος του LDR (0-1023)
  // σε ένα ακουστικό εύρος συχνοτήτων για το buzzer (π.χ. 150Hz έως 1500Hz)
  frequency = map(ldrValue, 0, 1023, 150, 1500);

  // 3. Στέλνουμε τη συχνότητα στο buzzer για να παραχθεί ο ήχος
  tone(buzzerPin, frequency);

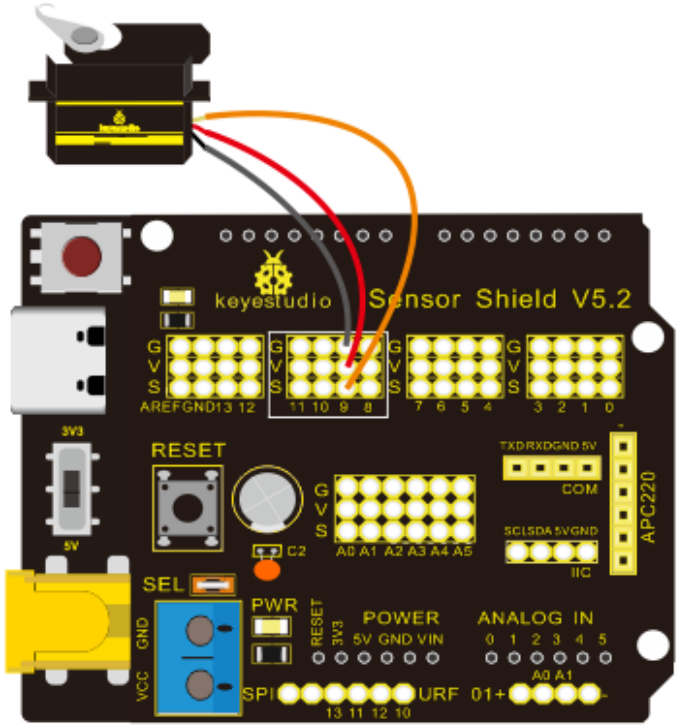
  // Εκτύπωση των τιμών στον υπολογιστή για έλεγχο και κατανόηση
  Serial.print("Φως (LDR): ");
  Serial.print(ldrValue);
  Serial.print("  ->  Συχνότητα (Hz): ");
  Serial.println(frequency);

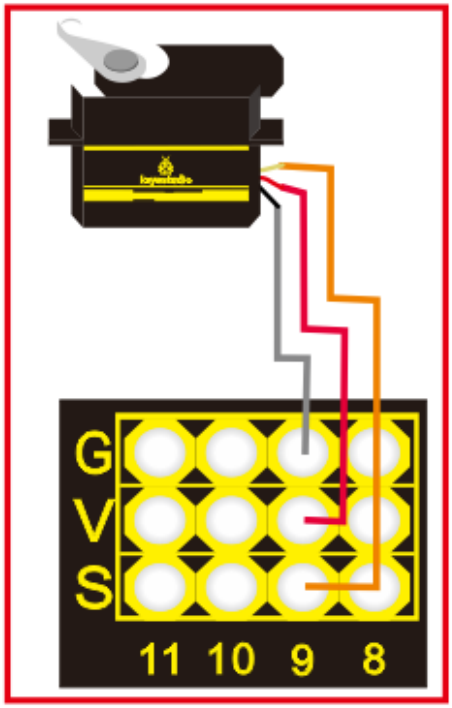
  delay(10); // Πολύ μικρή καθυστέρηση για άμεση και ομαλή ανταπόκριση στον ήχο
}
```

Lesson 05 - Servo Motors

Wiring

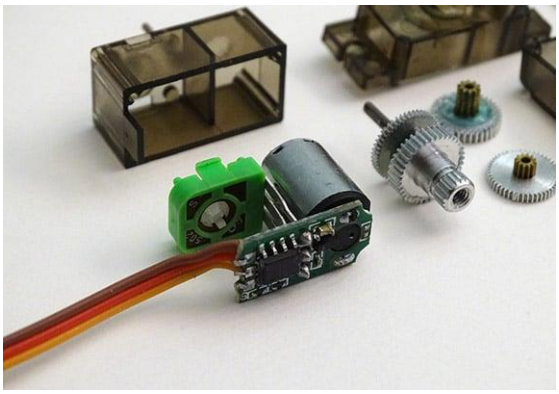
Connect the servo motor module to G, V, and D9.

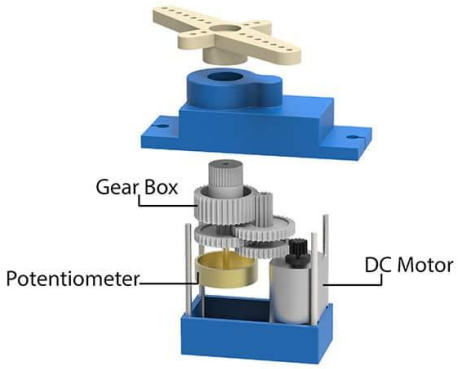




A servo motor can be used to move an object. In our Smart Home project, it can control doors and windows. In this lesson, we will introduce its basic operating principle and show how to use it.

A servo is a rotary mechanism that controls its position. It mainly consists of a case, an electronic circuit, an electric motor, gears, and a position sensor.

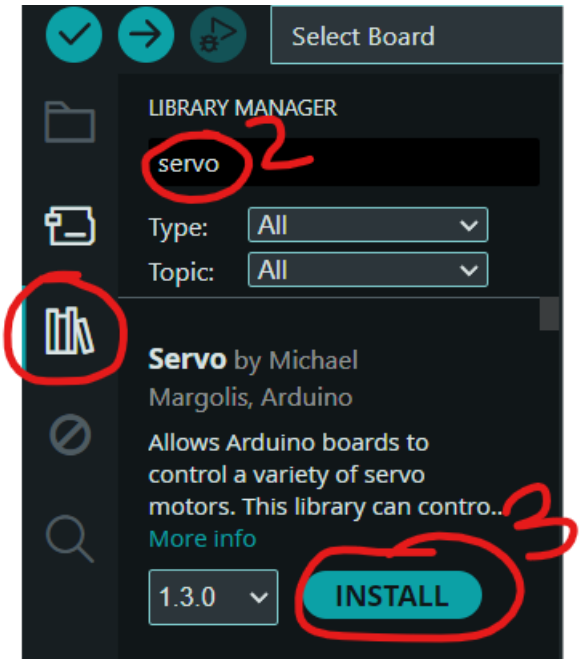




The servo receives a signal from the microcontroller (Arduino). Its internal mechanism compares the requested position with the position measured by its sensor. The motor moves until it reaches the requested position, then stops.
A servo can usually rotate from 0° to 180°.

Programming

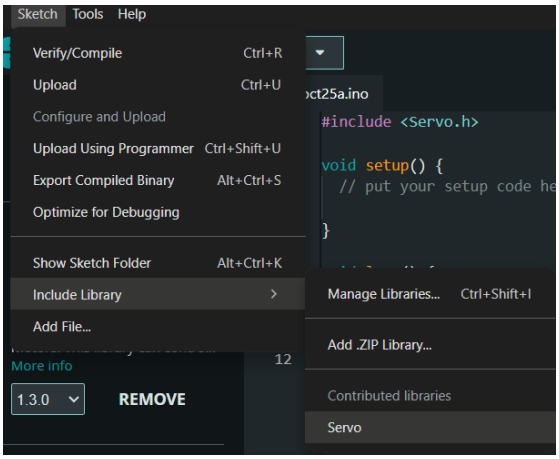
A servo is a motor whose position, or angle of rotation, can be set. To control it, we use the Servo.h library, which must be installed as shown on the right.



Next, include the library in your program in one of two ways: type the following at the beginning of the program:

```
#include <Servo.h>
```

Or, in the Arduino IDE, select:
Sketch > Include Library > Servo



To change the motor's angle, use the command:

```
myservo.write(angle);
```

As with an LED, we first need to set things up. For a servo, declare the motor as a variable near the beginning of the program by writing Servo myservo;
Then tell the Arduino which pin it is connected to by writing myservo.attach(PIN).

```
#include <Servo.h>
int pos = 90;
Servo myservo;

void setup() {
  myservo.attach(9);
  myservo.write(pos);

}
```

Example Program

The program moves the servo from 0° to 180° and then back to 0°.

```
#include <Servo.h>
```

```
Servo myservo;
int pos = 0;

void setup() {
  myservo.attach(9);
}

void loop() {
 // Move from 0° to 180°
  for (pos = 0; pos <= 180; pos++) {
    myservo.write(pos);
    delay(15);
  }
 // Move from 180° to 0°
  for (pos = 180; pos >= 0; pos--) {
    myservo.write(pos);
    delay(15);
  }
}
```

Check

Click Verify and then Upload. The servo should move smoothly from one side to the other and back.
If it does not work, check:

- That the code uploaded without errors.
- That the servo is connected to the correct pins: G, V, and D9.
- That the signal wire is connected to the pin specified in myservo.attach(9).

The Serial Monitor as an Input

The Serial Monitor lets us send information from the computer to the Arduino. So far, we have only received information from the Arduino. Now it is time to send something back! In this example, we type a number and the Arduino stores it and displays it again.

```
int number;

void setup() {
 Serial.begin(9600); // Start serial communication
```

```
}

void loop() {
 if (Serial.available() > 0) { // Check whether anything was received
  number = Serial.parseInt(); // Read the number

    Serial.print("The number is: ");
  Serial.println(number); // Display the number
  }
}
```

Challenge: Guess the Number!

The Arduino chooses a random secret number from 1 to 100. Try to guess it by typing numbers in the Serial Monitor. You have 10 attempts!

After each guess, the Arduino tells you whether the secret number is higher or lower. If you guess correctly, it displays a success message.

Challenge: Can you guess it in 7 attempts or fewer? Use binary search: each time, try a number near the middle of the remaining range.

Copy and fix the program below!

```
int secretNumber;
int guess;
int attempts = 0;

void setup() {
  Serial.begin(9600);
  randomSeed(analogRead(A0));
  secretNumber = random(1, 101);
  Serial.println("=== GUESS THE NUMBER ===");
```

```
  Serial.println("I have chosen a number from 1 to 100.");
  Serial.println("You have 10 attempts.");
  Serial.println();
}
void loop() {
  Serial.println("------------------------");
  Serial.print("Attempts left: ");
  Serial.println(10 - attempts);
  Serial.print("Enter your guess: ");
 while (Serial.available() == 0) {} // Wait for you to enter a number
  guess = Serial.parseInt();
 // Clear Enter and any remaining characters
  while (Serial.available() > 0) {Serial.read();}

  attempts++;
  Serial.print("Your guess: ");
  Serial.println(guess);

 if (check whether the guess matches the number) {
    Serial.println("Correct! You found the number!");
    Serial.println("A new game is starting.");
    attempts = 0;
    secretNumber = random(1, 101);
  }
 else if (check whether the guess is lower than the number) {
    Serial.println("Try a bigger number.");
  }
  else {
    Serial.println("Try a smaller number.");
  }
 if (check whether 10 attempts have been used and the guess is incorrect) {
    Serial.println();
    Serial.println("No attempts left!");
    Serial.print("The number was: ");
    Serial.println(secretNumber);
    Serial.println("A new game is starting.");
    attempts = 0;
    secretNumber = random(1, 101);
  }
  Serial.println();
```

Challenge 2

Write a program that lets you enter a number from 0 to 180 in the Serial Monitor and moves the servo to that angle.

Serial monitor input Servo Solution

```
#include <Servo.h>

Servo myservo;
int angle;

void setup() {
  Serial.begin(9600);
  myservo.attach(9);

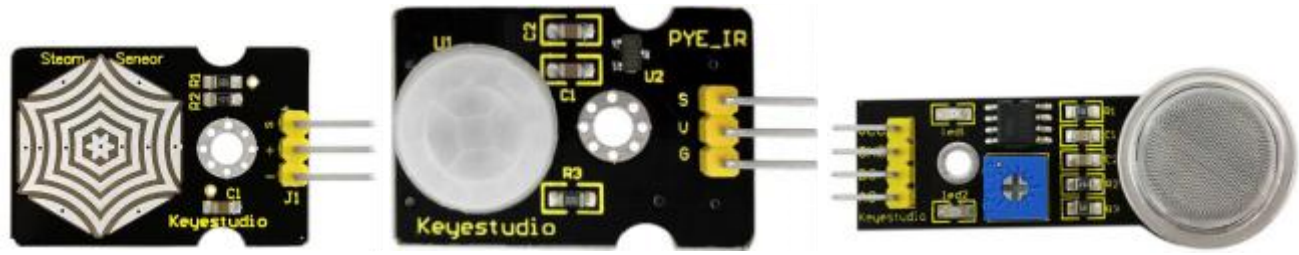
  Serial.println("Enter an angle from 0 to 180:");
}

void loop() {
  if (Serial.available() > 0) {
    angle = Serial.parseInt();

    if (angle >= 0 && angle <= 180) {
      myservo.write(angle);
      Serial.print("Servo angle: ");
      Serial.println(angle);
    } else {
      Serial.println("Enter a number from 0 to 180.");
    }

    while (Serial.available() > 0) {
      Serial.read();  // Καθαρίζει το Enter
    }
  }
}
```

Lesson 06 - Other Sensors (Water, PIR, Gas)



Water Sensor (1)

This sensor detects water on its surface. It has parallel exposed conductive lines. When water covers them, the electrical conductivity between the lines changes, which changes the sensor's analog output. It can be used as a rain detector or to detect water on a surface.
Caution: The sensor's connection area is not waterproof. Do not immerse the sensor in water.

PIR Motion Sensor (2)

The PIR motion sensor detects changes in infrared radiation in its surroundings, such as those caused by a moving person. When it detects motion, its digital output changes state.

Gas Sensor (3)

The MQ-2 gas sensor is used in systems for detecting flammable gases and smoke. It responds to gases such as liquefied petroleum gas (LPG), natural gas, hydrogen, alcohol, and benzene, as well as smoke. The sensor needs time to warm up before use. Its analog output changes depending on the presence and concentration of gases or smoke. The module's sensitivity can be adjusted by turning the potentiometer.

How will you connect each sensor, and which commands will you use to read their values?

.....

Challenge!

Build a complete home security system. If rain, an intruder, or a dangerous gas is detected, an alarm should sound!

Build instructions - You must complete the Serial Monitor Servo Input activity first.

https://docs.keyestudio.com/projects/KS0085/en/latest/docs/2.%20Product%20installation/2.%20Product%20installation.html